Developing with GSSAPI

The GSSAPI (Generic Security Services API) allows applications to communicate securely using Kerberos 5 or other security mechanisms. We recommend using the GSSAPI (or a higher-level framework which encompasses GSSAPI, such as SASL) for secure network communication over using the libkrb5 API directly.

GSSAPIv2 is specified in **RFC 2743** and **RFC 2744**. Also see **RFC 7546** for a description of how to use the GSSAPI in a client or server program.

This documentation will describe how various ways of using the GSSAPI will behave with the krb5 mechanism as implemented in MIT krb5, as well as krb5-specific extensions to the GSSAPI.

Name types

A GSSAPI application can name a local or remote entity by calling gss_import_name, specifying a name type and a value. The following name types are supported by the krb5 mechanism:

- **GSS_C_NT_HOSTBASED_SERVICE**: The value should be a string of the form `service` or `service@hostname`. This is the most common way to name target services when initiating a security context, and is the most likely name type to work across multiple mechanisms.
- **GSS_KRB5_NT_PRINCIPAL_NAME**: The value should be a principal name string. This name type only works with the krb5 mechanism, and is defined in the `<gssapi/gssapi_krb5.h>` header.
- **GSS_C_NT_USER_NAME** or **GSS_C_NULL_OID**: The value is treated as an unparsed principal name string, as above. These name types may work with mechanisms other than krb5, but will have different interpretations in those mechanisms. **GSS_C_NT_USER_NAME** is intended to be used with a local username, which will parse into a single-component principal in the default realm.
- **GSS_C_NT_ANONYMOUS**: The value is ignored. The anonymous principal is used, allowing a client to authenticate to a server without asserting a particular identity (which may or may not be allowed by a particular server or Kerberos realm).
- **GSS_C_NT_MACHINE_UID_NAME**: The value is uid_t object.

On this page

Developing with GSSAPI
Name types
Initiator credentials
Acceptor names
Name Attributes
Credential store extensions
Importing and exporting credentials
Constrained delegation (S4U)
Channel binding behavior and GSS_C_CHANNEL_BOU
AEAD message wrapping
IOV message wrapping
IOV MIC tokens

Table of contents

For users
For administrators
For application developers
Developing with GSSAPI
Year 2038 considerations for uses of krb5_timestamp
Differences between

On Unix-like systems, the username of the uid is looked up in the system user database and the resulting username is parsed as a principal name.

- **GSS_C_NT_STRING_UID_NAME**: As above, but the value is a decimal string representation of the uid.
- **GSS_C_NT_EXPORT_NAME**: The value must be the result of a gss_export_name call.
- **GSS_KRB5_NT_ENTERPRISE_NAME**: The value should be a krb5 enterprise name string (see **RFC 6806** section 5), in the form `user@suffix`. This name type is used to convey alias names, and is defined in the `<gssapi/gssapi_krb5.h>` header. (New in release 1.17.)
- **GSS_KRB5_NT_X509_CERT**: The value should be an X.509 certificate encoded according to **RFC 5280**. This name form can be used for the desired_name parameter of gss_acquire_cred_impersonate_name(), to identify the S4U2Self user by certificate. (New in release 1.19.)

Initiator credentials

A GSSAPI client application uses gss_init_sec_context to establish a security context. The *initiator_cred_handle* parameter determines what tickets are used to establish the connection. An application can either pass **GSS_C_NO_CREDENTIAL** to use the default client credential, or it can use gss_acquire_cred beforehand to acquire an initiator credential. The call to gss_acquire_cred may include a *desired_name* parameter, or it may pass **GSS_C_NO_NAME** if it does not have a specific name preference.

If the desired name for a krb5 initiator credential is a host-based name, it is converted to a principal name of the form `service/hostname` in the local realm, where *hostname* is the local hostname if not specified. The hostname will be canonicalized using forward name resolution, and possibly also using reverse name resolution depending on the value of the **rdns** variable in [libdefaults].

If a desired name is specified in the call to gss_acquire_cred, the krb5 mechanism will attempt to find existing tickets for that client principal name in the default credential cache or collection. If the default cache type does not support a collection, and the default cache contains credentials for a different principal than the desired name, a **GSS_S_CRED_UNAVAIL** error will be returned with a minor code indicating a mismatch.

If no existing tickets are available for the desired name, but the name has an entry in the default client keytab, the krb5 mechanism will acquire initial tickets for the name using the default client keytab.

If no desired name is specified, credential acquisition will be deferred until the credential is used in a call to gss_init_sec_context or

Heimdal and MIT Kerberos API
Initial credentials
Principal manipulation and parsing
Complete reference - API and datatypes
For plugin module developers
Building Kerberos V5
Kerberos V5 concepts
Protocols and file formats
MIT Kerberos features
How to build this documentation from the source
Contributing to the MIT Kerberos Documentation
Resources

Full Table of Contents

Search

Go

gss_inquire_cred. If the call is to gss_init_sec_context, the target name will be used to choose a client principal name using the credential cache selection facility. (This facility might, for instance, try to choose existing tickets for a client principal in the same realm as the target service). If there are no existing tickets for the chosen principal, but it is present in the default client keytab, the krb5 mechanism will acquire initial tickets using the keytab.

If the target name cannot be used to select a client principal (because the credentials are used in a call to gss_inquire_cred), or if the credential cache selection facility cannot choose a principal for it, the default credential cache will be selected if it exists and contains tickets.

If the default credential cache does not exist, but the default client keytab does, the krb5 mechanism will try to acquire initial tickets for the first principal in the default client keytab.

If the krb5 mechanism acquires initial tickets using the default client keytab, the resulting tickets will be stored in the default cache or collection, and will be refreshed by future calls to gss_acquire_cred as they approach their expire time.

Acceptor names

A GSSAPI server application uses gss_accept_sec_context to establish a security context based on tokens provided by the client. The *acceptor_cred_handle* parameter determines what keytab entries may be authenticated to by the client, if the krb5 mechanism is used.

The simplest choice is to pass **GSS_C_NO_CREDENTIAL** as the acceptor credential. In this case, clients may authenticate to any service principal in the default keytab (typically DEFKTNAME, or the value of the **KRB5_KTNAME** environment variable). This is the recommended approach if the server application has no specific requirements to the contrary.

A server may acquire an acceptor credential with gss_acquire_cred and a *cred_usage* of **GSS_C_ACCEPT** or **GSS_C_BOTH**. If the *desired_name* parameter is **GSS_C_NO_NAME**, then clients will be allowed to authenticate to any service principal in the default keytab, just as if no acceptor credential was supplied.

If a server wishes to specify a *desired_name* to gss_acquire_cred, the most common choice is a host-based name. If the host-based *desired_name* contains just a *service*, then clients will be allowed to authenticate to any host-based service principal (that is, a principal of the form `service/hostname@REALM`) for the named service, regardless of hostname or realm, as long as it is present in the default keytab. If the input name contains both a *service* and a *hostname*, clients will be allowed to authenticate to any host-based principal for the named service and hostname, regardless of realm.

> **Note**

> If a *hostname* is specified, it will be canonicalized using forward name resolution, and possibly also using reverse name resolution depending on the value of the **rdns** variable in [libdefaults].

> **Note**
>
> If the **ignore_acceptor_hostname** variable in [libdefaults] is enabled, then *hostname* will be ignored even if one is specified in the input name.

> **Note**
>
> In MIT krb5 versions prior to 1.10, and in Heimdal's implementation of the krb5 mechanism, an input name with just a *service* is treated like an input name of `service@localhostname`, where *localhostname* is the string returned by gethostname().

If the *desired_name* is a krb5 principal name or a local system name type which is mapped to a krb5 principal name, clients will only be allowed to authenticate to that principal in the default keytab.

Name Attributes

In release 1.8 or later, the gss_inquire_name and gss_get_name_attribute functions, specified in **RFC 6680**, can be used to retrieve name attributes from the *src_name* returned by gss_accept_sec_context. The following attributes are defined when the krb5 mechanism is used:

- "auth-indicators" attribute:

This attribute will be included in the gss_inquire_name output if the ticket contains authentication indicators. One indicator is returned per invocation of gss_get_name_attribute, so multiple invocations may be necessary to retrieve all of the indicators from the ticket. (New in release 1.15.)

Credential store extensions

Beginning with release 1.11, the following GSSAPI extensions declared in `<gssapi/gssapi_ext.h>` can be used to specify how credentials are acquired or stored:

```
struct gss_key_value_element_struct {
    const char *key;
    const char *value;
};
typedef struct gss_key_value_element_struct gss_key_value_element_desc;

struct gss_key_value_set_struct {
    OM_uint32 count;
    gss_key_value_element_desc *elements;
};
typedef const struct gss_key_value_set_struct gss_key_value_set_desc;
```

```
typedef const gss_key_value_set_desc *gss_const_key_value_set_t;

OM_uint32 gss_acquire_cred_from(OM_uint32 *minor_status,
                                const gss_name_t desired_name,
                                OM_uint32 time_req,
                                const gss_OID_set desired_mechs,
                                gss_cred_usage_t cred_usage,
                                gss_const_key_value_set_t cred_store,
                                gss_cred_id_t *output_cred_handle,
                                gss_OID_set *actual_mechs,
                                OM_uint32 *time_rec);

OM_uint32 gss_store_cred_into(OM_uint32 *minor_status,
                              gss_cred_id_t input_cred_handle,
                              gss_cred_usage_t cred_usage,
                              const gss_OID desired_mech,
                              OM_uint32 overwrite_cred,
                              OM_uint32 default_cred,
                              gss_const_key_value_set_t cred_store,
                              gss_OID_set *elements_stored,
                              gss_cred_usage_t *cred_usage_stored);
```

The additional *cred_store* parameter allows the caller to specify information about how the credentials should be obtained and stored. The following options are supported by the krb5 mechanism:

- **ccache**: For acquiring initiator credentials, the name of the credential cache to which the handle will refer. For storing credentials, the name of the cache or collection where the credentials will be stored (see below).
- **client_keytab**: For acquiring initiator credentials, the name of the keytab which will be used, if necessary, to refresh the credentials in the cache.
- **keytab**: For acquiring acceptor credentials, the name of the keytab to which the handle will refer. In release 1.19 and later, this option also determines the keytab to be used for verification when initiator credentials are acquired using a password and verified.
- **password**: For acquiring initiator credentials, this option instructs the mechanism to acquire fresh credentials into a unique memory credential cache. This option may not be used with the **ccache** or **client_keytab** options, and a *desired_name* must be specified. (New in release 1.19.)
- **rcache**: For acquiring acceptor credentials, the name of the replay cache to be used when processing the initiator tokens. (New in release 1.13.)
- **verify**: For acquiring initiator credentials, this option instructs the mechanism to verify the credentials by obtaining a ticket to a service with a known key. The service key is obtained from the keytab specified with the **keytab** option or the default keytab. The value may be the name of a principal in the keytab, or the empty string. If the empty string is given, any `host` service principal in the keytab may be used. (New in release 1.19.)

In release 1.20 or later, if a collection name is specified for **cache** in a call to gss_store_cred_into(), an existing cache for the client principal within the collection will be selected, or a new cache will be created within the collection. If *overwrite_cred* is false and the selected credential cache already exists, a **GSS_S_DUPLICATE_ELEMENT** error will be returned. If *default_cred* is true, the primary cache of the collection will be switched to the selected cache.

Importing and exporting credentials

The following GSSAPI extensions can be used to import and export credentials (declared in `<gssapi/gssapi_ext.h>`):

```
OM_uint32 gss_export_cred(OM_uint32 *minor_status,
                          gss_cred_id_t cred_handle,
                          gss_buffer_t token);

OM_uint32 gss_import_cred(OM_uint32 *minor_status,
                          gss_buffer_t token,
                          gss_cred_id_t *cred_handle);
```

The first function serializes a GSSAPI credential handle into a buffer; the second unseralizes a buffer into a GSSAPI credential handle. Serializing a credential does not destroy it. If any of the mechanisms used in *cred_handle* do not support serialization, gss_export_cred will return **GSS_S_UNAVAILABLE**. As with other GSSAPI serialization functions, these extensions are only intended to work with a matching implementation on the other side; they do not serialize credentials in a standardized format.

A serialized credential may contain secret information such as ticket session keys. The serialization format does not protect this information from eavesdropping or tampering. The calling application must take care to protect the serialized credential when communicating it over an insecure channel or to an untrusted party.

A krb5 GSSAPI credential may contain references to a credential cache, a client keytab, an acceptor keytab, and a replay cache. These resources are normally serialized as references to their external locations (such as the filename of the credential cache). Because of this, a serialized krb5 credential can only be imported by a process with similar privileges to the exporter. A serialized credential should not be trusted if it originates from a source with lower privileges than the importer, as it may contain references to external credential cache, keytab, or replay cache resources not accessible to the originator.

An exception to the above rule applies when a krb5 GSSAPI credential refers to a memory credential cache, as is normally the case for delegated credentials received by gss_accept_sec_context. In this case, the contents of the credential cache are serialized, so that the resulting token may be imported even if the original memory credential cache no longer exists.

Constrained delegation (S4U)

The Microsoft S4U2Self and S4U2Proxy Kerberos protocol extensions allow an intermediate service to acquire credentials from a client to a target service without requiring the client to delegate a ticket-granting ticket, if the KDC is configured to allow it.

To perform a constrained delegation operation, the intermediate service must submit to the KDC an "evidence ticket" from the client to the intermediate service. An evidence ticket can be acquired when the client authenticates to the intermediate service with Kerberos, or with an S4U2Self request if the KDC allows it. The MIT krb5 GSSAPI library represents an evidence ticket using a "proxy credential", which is a special kind of gss_cred_id_t object whose underlying credential cache contains the evidence ticket and a krbtgt ticket for the intermediate service.

To acquire a proxy credential during client authentication, the service should first create an acceptor credential using the **GSS_C_BOTH** usage. The application should then pass this credential as the *acceptor_cred_handle* to gss_accept_sec_context, and also pass a *delegated_cred_handle* output parameter to receive a proxy credential containing the evidence ticket. The output value of *delegated_cred_handle* may be a delegated ticket-granting ticket if the client sent one, or a proxy credential if not. If the library can determine that the client's ticket is not a valid evidence ticket, it will place **GSS_C_NO_CREDENTIAL** in *delegated_cred_handle*.

To acquire a proxy credential using an S4U2Self request, the service can use the following GSSAPI extension:

```
OM_uint32 gss_acquire_cred_impersonate_name(OM_uint32 *minor_status,
                                            gss_cred_id_t icred,
                                            gss_name_t desired_name,
                                            OM_uint32 time_req,
                                            gss_OID_set desired_mechs,
                                            gss_cred_usage_t cred_usage,
                                            gss_cred_id_t *output_cred,
                                            gss_OID_set *actual_mechs,
                                            OM_uint32 *time_rec);
```

The parameters to this function are similar to those of gss_acquire_cred, except that *icred* is used to make an S4U2Self request to the KDC for a ticket from *desired_name* to the intermediate service. Both *icred* and *desired_name* are required for this function; passing **GSS_C_NO_CREDENTIAL** or **GSS_C_NO_NAME** will cause the call to fail. *icred* must contain a krbtgt ticket for the intermediate service. The result of this operation is a proxy credential. (Prior to release 1.18, the result of this operation may be a regular credential for *desired_name*, if the KDC issues a non-forwardable ticket.)

Once the intermediate service has a proxy credential, it can simply pass it to gss_init_sec_context as the *initiator_cred_handle* parameter, and the desired service as the *target_name* parameter. The GSSAPI library will present the krbtgt ticket and evidence ticket in the proxy credential to the KDC in an S4U2Proxy request; if the intermediate

service has the appropriate permissions, the KDC will issue a ticket from the client to the target service. The GSSAPI library will then use this ticket to authenticate to the target service.

If an application needs to find out whether a credential it holds is a proxy credential and the name of the intermediate service, it can query the credential with the **GSS_KRB5_GET_CRED_IMPERSONATOR** OID (new in release 1.16, declared in `<gssapi/gssapi_krb5.h>`) using the gss_inquire_cred_by_oid extension (declared in `<gssapi/gssapi_ext.h>`):

```
OM_uint32 gss_inquire_cred_by_oid(OM_uint32 *minor_status,
                                  const gss_cred_id_t cred_handle,
                                  gss_OID desired_object,
                                  gss_buffer_set_t *data_set);
```

If the call succeeds and *cred_handle* is a proxy credential, *data_set* will be set to a single-element buffer set containing the unparsed principal name of the intermediate service. If *cred_handle* is not a proxy credential, *data_set* will be set to an empty buffer set. If the library does not support the query, gss_inquire_cred_by_oid will return **GSS_S_UNAVAILABLE**.

Channel binding behavior and GSS_C_CHANNEL_BOUND_FLAG

GSSAPI channel bindings can be used to limit the scope of a context establishment token to a particular protected channel or endpoint, such as a TLS channel or server certificate. Channel bindings can be supplied via the *input_chan_bindings* parameter to either gss_init_sec_context() or gss_accept_sec_context().

If both the initiator and acceptor of a GSSAPI exchange supply matching channel bindings, **GSS_C_CHANNEL_BOUND_FLAG** will be included in the gss_accept_sec_context() *ret_flags* result. If either the initiator or acceptor (or both) do not supply channel bindings, the exchange will succeed, but **GSS_C_CHANNEL_BOUND_FLAG** will not be included in the return flags. If the acceptor and initiator both inlude channel bindings but they do not match, the exchange will fail.

If **GSS_C_CHANNEL_BOUND_FLAG** is included in the *req_flags* parameter of gss_init_sec_context(), the initiator will add the Microsoft KERB_AP_OPTIONS_CBT extension to the Kerberos authenticator. This extension requests that the acceptor strictly enforce channel bindings, causing the exchange to fail if the acceptor supplies channel bindings and the initiator does not. The KERB_AP_OPTIONS_CBT extension will also be included if the **client_aware_channel_bindings** variable is set to `true` in [libdefaults].

Prior to release 1.19, **GSS_C_CHANNEL_BOUND_FLAG** is not implemented, and the exchange will fail if the acceptor supply channel bindings and the initiator does not (but not vice versa). Between

releases 1.19 and 1.21, **GSS_C_CHANNEL_BOUND_FLAG** is not recognized as an initiator flag, so **client_aware_channel_bindings** is the only way to cause KERB_AP_OPTIONS_CBT to be included.

AEAD message wrapping

The following GSSAPI extensions (declared in `<gssapi/gssapi_ext.h>`) can be used to wrap and unwrap messages with additional "associated data" which is integrity-checked but is not included in the output buffer:

```
OM_uint32 gss_wrap_aead(OM_uint32 *minor_status,
                        gss_ctx_id_t context_handle,
                        int conf_req_flag, gss_qop_t qop_req,
                        gss_buffer_t input_assoc_buffer,
                        gss_buffer_t input_payload_buffer,
                        int *conf_state,
                        gss_buffer_t output_message_buffer);

OM_uint32 gss_unwrap_aead(OM_uint32 *minor_status,
                          gss_ctx_id_t context_handle,
                          gss_buffer_t input_message_buffer,
                          gss_buffer_t input_assoc_buffer,
                          gss_buffer_t output_payload_buffer,
                          int *conf_state,
                          gss_qop_t *qop_state);
```

Wrap tokens created with gss_wrap_aead will successfully unwrap only if the same *input_assoc_buffer* contents are presented to gss_unwrap_aead.

IOV message wrapping

The following extensions (declared in `<gssapi/gssapi_ext.h>`) can be used for in-place encryption, fine-grained control over wrap token layout, and for constructing wrap tokens compatible with Microsoft DCE RPC:

```
typedef struct gss_iov_buffer_desc_struct {
    OM_uint32 type;
    gss_buffer_desc buffer;
} gss_iov_buffer_desc, *gss_iov_buffer_t;

OM_uint32 gss_wrap_iov(OM_uint32 *minor_status,
                       gss_ctx_id_t context_handle,
                       int conf_req_flag, gss_qop_t qop_req,
                       int *conf_state,
                       gss_iov_buffer_desc *iov, int iov_count);

OM_uint32 gss_unwrap_iov(OM_uint32 *minor_status,
                         gss_ctx_id_t context_handle,
                         int *conf_state, gss_qop_t *qop_state,
                         gss_iov_buffer_desc *iov, int iov_count);

OM_uint32 gss_wrap_iov_length(OM_uint32 *minor_status,
                              gss_ctx_id_t context_handle,
                              int conf_req_flag,
                              gss_qop_t qop_req, int *conf_state,
                              gss_iov_buffer_desc *iov,
                              int iov_count);
```

```
OM_uint32 gss_release_iov_buffer(OM_uint32 *minor_status,
                                 gss_iov_buffer_desc *iov,
                                 int iov_count);
```

The caller of gss_wrap_iov provides an array of gss_iov_buffer_desc structures, each containing a type and a gss_buffer_desc structure. Valid types include:

- **GSS_C_BUFFER_TYPE_DATA**: A data buffer to be included in the token, and to be encrypted or decrypted in-place if the token is confidentiality-protected.
- **GSS_C_BUFFER_TYPE_HEADER**: The GSSAPI wrap token header and underlying cryptographic header.
- **GSS_C_BUFFER_TYPE_TRAILER**: The cryptographic trailer, if one is required.
- **GSS_C_BUFFER_TYPE_PADDING**: Padding to be combined with the data during encryption and decryption. (The implementation may choose to place padding in the trailer buffer, in which case it will set the padding buffer length to 0.)
- **GSS_C_BUFFER_TYPE_STREAM**: For unwrapping only, a buffer containing a complete wrap token in standard format to be unwrapped.
- **GSS_C_BUFFER_TYPE_SIGN_ONLY**: A buffer to be included in the token's integrity protection checksum, but not to be encrypted or included in the token itself.

For gss_wrap_iov, the IOV list should contain one HEADER buffer, followed by zero or more SIGN_ONLY buffers, followed by one or more DATA buffers, followed by a TRAILER buffer. The memory pointed to by the buffers is not required to be contiguous or in any particular order. If *conf_req_flag* is true, DATA buffers will be encrypted in-place, while SIGN_ONLY buffers will not be modified.

The type of an output buffer may be combined with **GSS_C_BUFFER_FLAG_ALLOCATE** to request that gss_wrap_iov allocate the buffer contents. If gss_wrap_iov allocates a buffer, it sets the **GSS_C_BUFFER_FLAG_ALLOCATED** flag on the buffer type. gss_release_iov_buffer can be used to release all allocated buffers within an iov list and unset their allocated flags. Here is an example of how gss_wrap_iov can be used with allocation requested (*ctx* is assumed to be a previously established gss_ctx_id_t):

```
OM_uint32 major, minor;
gss_iov_buffer_desc iov[4];
char str[] = "message";

iov[0].type = GSS_IOV_BUFFER_TYPE_HEADER | GSS_IOV_BUFFER_FLAG_ALLOCATE;
iov[1].type = GSS_IOV_BUFFER_TYPE_DATA;
iov[1].buffer.value = str;
iov[1].buffer.length = strlen(str);
iov[2].type = GSS_IOV_BUFFER_TYPE_PADDING | GSS_IOV_BUFFER_FLAG_ALLOCATE;
iov[3].type = GSS_IOV_BUFFER_TYPE_TRAILER | GSS_IOV_BUFFER_FLAG_ALLOCATE;
```

```
major = gss_wrap_iov(&minor, ctx, 1, GSS_C_QOP_DEFAULT, NULL,
                     iov, 4);
if (GSS_ERROR(major))
    handle_error(major, minor);

/* Transmit or otherwise use resulting buffers. */

(void)gss_release_iov_buffer(&minor, iov, 4);
```

If the caller does not choose to request buffer allocation by gss_wrap_iov, it should first call gss_wrap_iov_length to query the lengths of the HEADER, PADDING, and TRAILER buffers. DATA buffers must be provided in the iov list so that padding length can be computed correctly, but the output buffers need not be initialized. Here is an example of using gss_wrap_iov_length and gss_wrap_iov:

```
OM_uint32 major, minor;
gss_iov_buffer_desc iov[4];
char str[1024] = "message", *ptr;

iov[0].type = GSS_IOV_BUFFER_TYPE_HEADER;
iov[1].type = GSS_IOV_BUFFER_TYPE_DATA;
iov[1].buffer.value = str;
iov[1].buffer.length = strlen(str);

iov[2].type = GSS_IOV_BUFFER_TYPE_PADDING;
iov[3].type = GSS_IOV_BUFFER_TYPE_TRAILER;

major = gss_wrap_iov_length(&minor, ctx, 1, GSS_C_QOP_DEFAULT,
                            NULL, iov, 4);
if (GSS_ERROR(major))
    handle_error(major, minor);
if (strlen(str) + iov[0].buffer.length + iov[2].buffer.length +
    iov[3].buffer.length > sizeof(str))
    handle_out_of_space_error();
ptr = str + strlen(str);
iov[0].buffer.value = ptr;
ptr += iov[0].buffer.length;
iov[2].buffer.value = ptr;
ptr += iov[2].buffer.length;
iov[3].buffer.value = ptr;

major = gss_wrap_iov(&minor, ctx, 1, GSS_C_QOP_DEFAULT, NULL,
                     iov, 4);
if (GSS_ERROR(major))
    handle_error(major, minor);
```

If the context was established using the **GSS_C_DCE_STYLE** flag (described in **RFC 4757**), wrap tokens compatible with Microsoft DCE RPC can be constructed. In this case, the IOV list must include a SIGN_ONLY buffer, a DATA buffer, a second SIGN_ONLY buffer, and a HEADER buffer in that order (the order of the buffer contents remains arbitrary). The application must pad the DATA buffer to a multiple of 16 bytes as no padding or trailer buffer is used.

gss_unwrap_iov may be called with an IOV list just like one which would be provided to gss_wrap_iov. DATA buffers will be decrypted in-place if they were encrypted, and SIGN_ONLY buffers will not be modified.

Alternatively, gss_unwrap_iov may be called with a single STREAM

buffer, zero or more SIGN_ONLY buffers, and a single DATA buffer. The STREAM buffer is interpreted as a complete wrap token. The STREAM buffer will be modified in-place to decrypt its contents. The DATA buffer will be initialized to point to the decrypted data within the STREAM buffer, unless it has the **GSS_C_BUFFER_FLAG_ALLOCATE** flag set, in which case it will be initialized with a copy of the decrypted data. Here is an example (*token* and *token_len* are assumed to be a pre-existing pointer and length for a modifiable region of data):

```
OM_uint32 major, minor;
gss_iov_buffer_desc iov[2];

iov[0].type = GSS_IOV_BUFFER_TYPE_STREAM;
iov[0].buffer.value = token;
iov[0].buffer.length = token_len;
iov[1].type = GSS_IOV_BUFFER_TYPE_DATA;
major = gss_unwrap_iov(&minor, ctx, NULL, NULL, iov, 2);
if (GSS_ERROR(major))
    handle_error(major, minor);

/* Decrypted data is in iov[1].buffer, pointing to a subregion of
 * token. */
```

IOV MIC tokens

The following extensions (declared in <gssapi/gssapi_ext.h>) can be used in release 1.12 or later to construct and verify MIC tokens using an IOV list:

```
OM_uint32 gss_get_mic_iov(OM_uint32 *minor_status,
                          gss_ctx_id_t context_handle,
                          gss_qop_t qop_req,
                          gss_iov_buffer_desc *iov,
                          int iov_count);

OM_uint32 gss_get_mic_iov_length(OM_uint32 *minor_status,
                                 gss_ctx_id_t context_handle,
                                 gss_qop_t qop_req,
                                 gss_iov_buffer_desc *iov,
                                 iov_count);

OM_uint32 gss_verify_mic_iov(OM_uint32 *minor_status,
                             gss_ctx_id_t context_handle,
                             gss_qop_t *qop_state,
                             gss_iov_buffer_desc *iov,
                             int iov_count);
```

The caller of gss_get_mic_iov provides an array of gss_iov_buffer_desc structures, each containing a type and a gss_buffer_desc structure. Valid types include:

- **GSS_C_BUFFER_TYPE_DATA** and **GSS_C_BUFFER_TYPE_SIGN_ONLY**: The corresponding buffer for each of these types will be signed for the MIC token, in the order provided.
- **GSS_C_BUFFER_TYPE_MIC_TOKEN**: The GSSAPI MIC token.

The type of the MIC_TOKEN buffer may be combined with **GSS_C_BUFFER_FLAG_ALLOCATE** to request that gss_get_mic_iov allocate the buffer contents. If gss_get_mic_iov allocates the buffer, it sets the **GSS_C_BUFFER_FLAG_ALLOCATED** flag on the buffer type. gss_release_iov_buffer can be used to release all allocated buffers within an iov list and unset their allocated flags. Here is an example of how gss_get_mic_iov can be used with allocation requested (*ctx* is assumed to be a previously established gss_ctx_id_t):

```
OM_uint32 major, minor;
gss_iov_buffer_desc iov[3];

iov[0].type = GSS_IOV_BUFFER_TYPE_DATA;
iov[0].buffer.value = "sign1";
iov[0].buffer.length = 5;
iov[1].type = GSS_IOV_BUFFER_TYPE_SIGN_ONLY;
iov[1].buffer.value = "sign2";
iov[1].buffer.length = 5;
iov[2].type = GSS_IOV_BUFFER_TYPE_MIC_TOKEN | GSS_IOV_BUFFER_FLAG_ALLOCAT

major = gss_get_mic_iov(&minor, ctx, GSS_C_QOP_DEFAULT, iov, 3);
if (GSS_ERROR(major))
    handle_error(major, minor);

/* Transmit or otherwise use iov[2].buffer. */

(void)gss_release_iov_buffer(&minor, iov, 3);
```

If the caller does not choose to request buffer allocation by gss_get_mic_iov, it should first call gss_get_mic_iov_length to query the length of the MIC_TOKEN buffer. Here is an example of using gss_get_mic_iov_length and gss_get_mic_iov:

```
OM_uint32 major, minor;
gss_iov_buffer_desc iov[2];
char data[1024];

iov[0].type = GSS_IOV_BUFFER_TYPE_MIC_TOKEN;
iov[1].type = GSS_IOV_BUFFER_TYPE_DATA;
iov[1].buffer.value = "message";
iov[1].buffer.length = 7;

major = gss_get_mic_iov_length(&minor, ctx, GSS_C_QOP_DEFAULT, iov, 2);
if (GSS_ERROR(major))
    handle_error(major, minor);
if (iov[0].buffer.length > sizeof(data))
    handle_out_of_space_error();
iov[0].buffer.value = data;

major = gss_get_mic_iov(&minor, ctx, GSS_C_QOP_DEFAULT, iov, 2);
if (GSS_ERROR(major))
    handle_error(major, minor);
```


© Copyright 1985-2025, MIT.